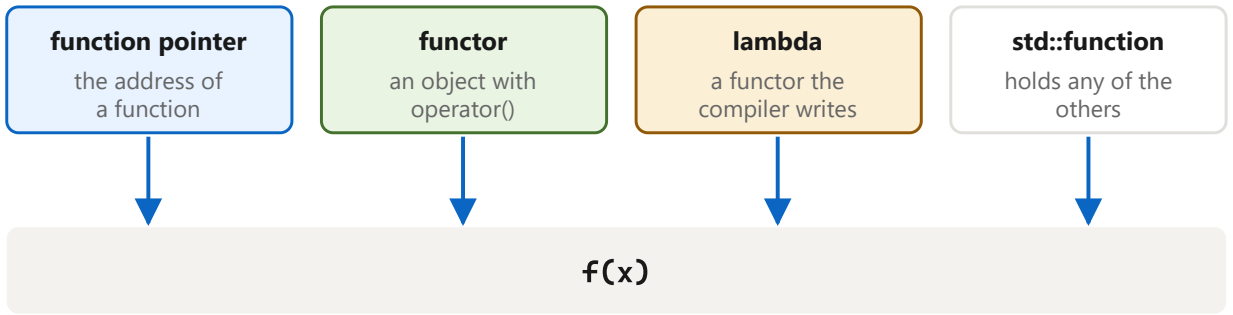
Function Pointers and Functors

The things you can call in C++: function pointers, functors and std::function, what each one can hold, and when to use which. Lambdas, the shortest way to write most of them, come next.

Something you can call

Many functions take "something to call": `std::sort` takes a comparison, `std::thread` takes a task, a library takes a callback. C++ has several kinds of callable. They look the same at the call site, `f(x)`, but they are different things underneath.



Four ways to hand over behavior; the call looks identical.

This lesson covers function pointers, functors and `std::function`. Lambdas come next and are the shortest way to write most of them.

Function pointers

Code lives in memory too, so a function has an address. A **function pointer** holds it, and calling through the pointer runs the function.



f(21) jumps to the address in f and runs twice(21)

A function pointer is just an address, 8 bytes on a 64-bit machine.

```
int twice(int x) { return 2 * x; }

// f points to a function: takes an int, returns an int
int (*f)(int) = twice;
std::cout << f(21);          // 42: the call goes through the pointer

using Op = int (*)(int);     // an alias makes the syntax readable
Op g = twice;
```

Passing a function pointer lets the caller choose the behavior at run time:

```
void apply(int (*op)(int), int x) { std::cout << op(x); }
apply(twice, 21);                                    // 42

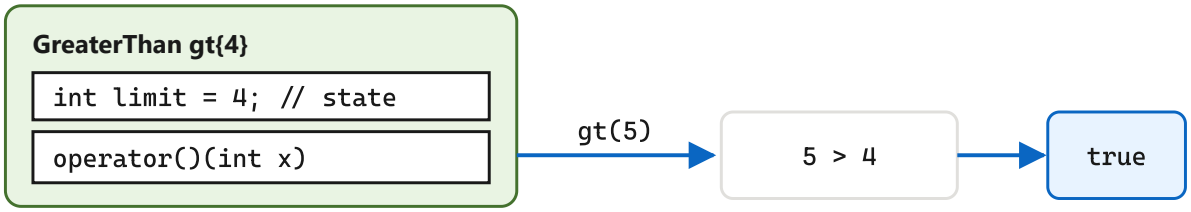
bool descending(int a, int b) { return a > b; }
// 9 5 2 1: the comparison is a function
std::sort(v.begin(), v.end(), descending);
```

- The pointer can point to any function with the **same signature**, and it can be changed while the program runs.
- It has **no state**: the function sees only its arguments and globals. Giving `descending` a "limit" would need a global variable.
- The call goes through the pointer, so the compiler usually cannot inline it.

C CALLBACKS C libraries pass state with a `void*` argument next to the function pointer, because a plain function cannot carry any. Functors and lambdas remove the need for it.

Functors: objects you can call

A **functor** (function object) is a class with an `operator()`. An object of it is called like a function, and it can **carry state** in its members.



The limit lives inside the object, so the call needs only the number to test.

```
struct GreaterThan {
    int limit;                                          // the state
    bool operator()(int x) const { return x > limit; }
};

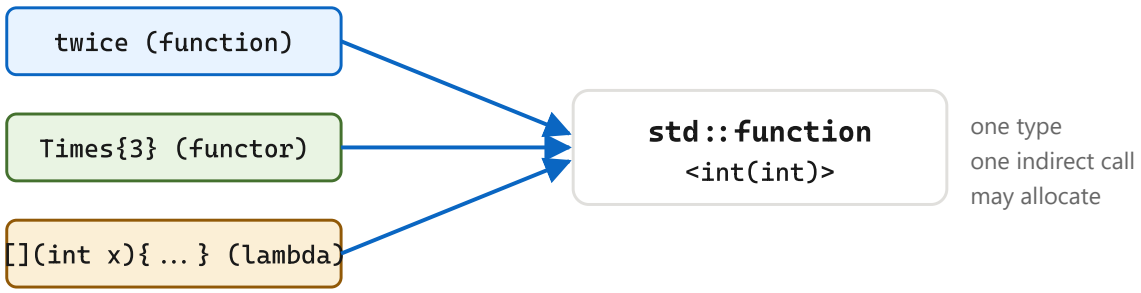
GreaterThan gt{4};
gt(5);                                                  // true
gt(3);                                                  // false
// counts the elements above 1
std::count_if(v.begin(), v.end(), GreaterThan{1});
```

- Each functor class is its own type, so a template such as `std::count_if` knows exactly which `operator()` runs and can **inline** it.
- The cost: a whole class, written away from where it is used, for one small piece of logic.
- The standard library ships ready-made ones: `std::less<>`, `std::greater<>`, `std::plus<>`. For example `std::sort(v.begin(), v.end(), std::greater<>{})` sorts descending.

SO WHAT IS A LAMBDA? A lambda is a functor the compiler writes for you, in place: `[limit](int x) { return x > limit; }` is `GreaterThan` with `limit` as its member. See Lambdas.

`std::function` and `std::invoke`

A function pointer, a functor and a lambda are three different types. `std::function<int(int)>` is one type that can hold **any** callable taking an `int` and returning an `int`. That is called **type erasure**.



Anything with the right signature fits; the price is an indirect call and sometimes a heap allocation.

```
struct Times { int k; int operator()(int x) const { return x * k; } };

std::function<int(int)> f = twice;          // a function pointer
std::cout << f(10);                         // 20
f = Times{3};                               // now a functor
std::cout << f(10);                         // 30
f = [](int x) { return x + 1; };            // now a lambda
std::cout << f(10);                         // 11
```

- It can hold callables of different types in one container, or keep one behind an interface.
- It may **allocate** when the callable does not fit its small internal buffer. Compiled with libstdc++: a callable holding one `int` allocated nothing, one holding a 64-byte array allocated once.

`std::invoke`

`std::invoke(f, args...)` calls anything callable, including a pointer to a **member function**, which a plain `f(x)` cannot do:

```
struct Account { int balance() const; };

Account acct;
std::invoke(&Account::balance, acct);       // calls acct.balance()
```

`std::bind`: know it, prefer a lambda

```
using namespace std::placeholders;
// fix the first argument: f(2, 3) is add3(1, 2, 3)
auto f = std::bind(add3, 1, _1, _2);
// the same, and easier to read
auto h = [](int b, int c) { return add3(1, b, c); };
```

`std::bind` predates lambdas (C++11). It copies its arguments, hides overloads and gives unreadable errors. You will see it in older code, so be able to read it; in new code write a lambda.

Which one when

KIND	HOLDS STATE?	INLINABLE?	OVERHEAD
function pointer	no	rarely (indirect call)	8 bytes
functor (class with `operator()`)	yes	yes: type known at compile time	size of its members
lambda	yes (captures)	yes: it is a functor	size of its captures
`std::function<R(Args...)>`	yes	no: type-erased	indirect call, and may allocate

```cpp
template <typename F>
// keeps the exact type: fully inlined
void apply(F f) { f(1); }

// accepts anything, one indirect call
void apply2(std::function<void(int)> f) { f(1); }
```

- **Take a template parameter** when you call it right away (`std::sort` does). Use `std::function` only when you must **store** callables of different types, or keep them behind an interface.
- Use a plain function pointer for C APIs. A lambda with no captures converts to one.
- Writing a functor by hand is now rare: a lambda is the same thing in one line.

> KEY POINT "Lambda vs `std::function`?" A lambda has its own type and is inlined; `std::function` erases the type, costs an indirect call and maybe an allocation. Use it only when you must store different callables in one place.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Function pointers	C++ Primer	6.7 Pointers to functions	276 to 281
Functors: objects you can call	C++ Primer	6.7 Pointers to functions	276 to 281
	C++ Primer	14.8 The function-call operator	600 to 607
std::function and std::invoke	C++ Primer	14.8 The function-call operator	600 to 607